

Guía Profesional de Instalación y Personalización de Arch Linux con RiceInstall

#IWCTL

Arch Linux Installation and Customization Manual

Complete Guide to Installation, Rice Configuration and Advanced System Setup

Version: 1.0

Date: May 2026

Platform: Arch Linux

Environment: BSPWM + Kitty + NvChad + Zsh

Introducción

Esta documentación describe de manera estructurada y profesional el proceso completo de instalación, configuración y personalización de Arch Linux. El objetivo es proporcionar un procedimiento reproducible que abarque desde la conexión inicial a Internet y la instalación del sistema base, hasta la implementación de un entorno gráfico altamente personalizado con BSPWM, la configuración avanzada del emulador de terminal Kitty, la integración de NvChad y el uso de herramientas de administración de paquetes como Pacman, Yay y Paru.

El documento está organizado en secciones independientes que explican tanto el propósito de cada configuración como los comandos necesarios para implementarla. Esta estructura permite utilizar la guía como manual de referencia personal, material de estudio o base para un entorno de trabajo profesional orientado al desarrollo de software, la administración de sistemas y la ciberseguridad.

Requisitos Previos

Antes de iniciar el proceso de instalación, es necesario contar con los siguientes elementos:

- Una memoria USB booteable creada con herramientas como Ventoy o Rufus.
 - La imagen ISO oficial de Arch Linux.
 - Acceso a una red con conexión a Internet.
 - Conocimientos básicos del uso de la terminal (de igual manera se va a explicar todo)
 - Tiempo suficiente para realizar la instalación y personalización del sistema.
-

Conexión a Internet con `iwctl`

Al iniciar el medio de instalación de Arch Linux, se debe seleccionar la opción **Boot in normal mode**. Una vez cargado el entorno en vivo desde la USB y la distro .iso, es necesario configurar la red inalámbrica utilizando `iwctl`, una interfaz interactiva en modo texto que permite detectar dispositivos Wi-Fi, escanear redes disponibles y establecer la conexión correspondiente.

- El nombre de la interfaz inalámbrica puede variar (`wlan0`, `wlp2s0`, entre otros).
- El comando `station wlan0 scan` no muestra salida inmediata, pero inicia el escaneo en segundo plano.
- Si se utiliza una máquina virtual con conectividad ya proporcionada por el host, este procedimiento puede no ser necesario.

```
> Boot in normal mode
> iwctl
```

`iwctl` para abrir la interfaz grafica de la instalación de la Wifi.

```
> device list
```

(tiene que ser el que te aparezca, sea wlan0 u otra cosa)

```
> station wlan0 scan
> station wlan0 get-networks
> station wlan0 connect [Nombre de la red escrita a mano]
> [Contraseña]
> exit
```

En el caso de wlan0, pueden tener otro sistema de wifi o ethernet, ya dicho anteriormente en device list.

A su vez, en el caso de "> station wlan0 scan" no aparecerá nada pero ya estará haciendo los escaneos.

Una vez terminado todo el proceso, ejecuta.

```
> archinstall
```

Esto abrirá un panel de instalación en el cual es importante que todo este en orden a la hora de instalar.

IMPORTANTE!!

Es importante que al momento de ejecutar el *archinstall* ya este toda la interfaz de red seleccionada, de lo contrario, no se podrá conectar a internet, a su vez si es en una maquina virtual, todo cambia, debido a que no es la misma configuración.

Si se usa una maquina virtual y el host de la maquina ya tiene internet, simplemente ejecutar *archinstall*, después en la configuración de redes de la interfaz, seleccionar la opción que esta detrás del host.

En caso de hacerlo como OS principal, seleccionar la configuración ISO de la maquina que ya trae

RiceInstall

Después de completar la instalación del sistema y acceder por primera vez, se instala LightDM para proporcionar una pantalla gráfica de autenticación. Posteriormente, se ejecuta RiceInstaller, un script que automatiza la configuración de temas, paneles, atajos y utilidades del entorno.

Una vez dentro del sistema, tienes que hacer el login y la contraseña que anteriormente pusiste. Para después aplicar una interfaz de inicio de sesión para mejor GUI

```
> sudo pacman -S lightdm lightdm-gtk-greeter
%% Habilitamos con: %%
> sudo systemctl enable lightdm.service
```

Se crea un symlink, y después instalamos el riceinstaller

```
> curl -LO https://gh0stzk.github.io/dotfiles/RiceInstaller
> chmod +x RiceInstaller
> ./RiceInstaller
<> y
<> *Decir que no al neovim, ya que vamos a instalar nvchad posteriormente
*
```

Una vez accedido en el directorio de Rice, es normal que se vea pequeño, primero se corrigen algunos errores.

```
> nvim ~/.zshrc
> * ESC *
> * :116 *
> * Comentar con '#' toda la linea *
```

En caso de estar en una maquina virtual VMware y no tener la interfaz (resolución) correcta:

```
> sudo pacman -S open-vm-tools
> sudo systemctl enable vmtoolsd.service
> sudo systemctl enable vmware-vmblock-fuse.service

> reboot
```

Después se reinicia.

NVchad

```
*Usuario normal*

> cd
> git clone https://github.com/NvChad/starter ~/.config/nvim && nvim

*root*
> cd
> git clone https://github.com/NvChad/starter ~/.config/nvim && nvim
```

Al abrir una terminal siempre se va a poner un dibujo seleccionado de la carpeta de configuración de colorscript, para poner un dibujo sin necesidad de abrir otra terminal

```
> colorscript -r
```

Esto se quita comentando en el mismo archivo `nvim ~/.zshrc` comentando con: `#` en AUTOSTART, la línea que dice: `$HOME/.local/bin/colorsript -r` quedando como:

```
# $HOME/.local/bin/colorsript -r
```

Configuración del Emulador de Terminal

En el entorno instalado mediante RiceInstaller se incluye, por defecto, Alacritty como emulador de terminal principal. Un emulador de terminal es la aplicación encargada de proporcionar la interfaz gráfica desde la cual se ejecuta la *shell* del sistema, como Zsh o `bash`. Es importante distinguir ambos conceptos: el emulador de terminal se encarga de mostrar la ventana, renderizar texto, gestionar pestañas, fuentes, transparencias y atajos de teclado; mientras que la shell interpreta los comandos introducidos por el usuario y los ejecuta en el sistema operativo.

Dentro del ecosistema GNU/Linux existen varios emuladores de terminal altamente populares, entre los que destacan Kitty, Alacritty y st (*simple terminal*). Todos cumplen la misma función fundamental, pero difieren en filosofía de diseño, capacidad de personalización y conjunto de funcionalidades.

Kitty es un emulador de terminal moderno y muy completo, acelerado por GPU, con soporte nativo para pestañas, divisiones de ventanas, transparencia, ligaduras tipográficas, portapapeles múltiples y una extensa configuración mediante archivos de texto. Está diseñado para usuarios avanzados que desean un entorno altamente personalizable y visualmente sofisticado, siendo especialmente popular entre desarrolladores y administradores de sistemas.

Alacritty comparte con Kitty el uso de aceleración por GPU y un enfoque en el rendimiento, pero mantiene una filosofía más minimalista. Ofrece una experiencia extremadamente rápida y estable, aunque con menos funciones integradas, ya que delega ciertas características avanzadas a herramientas externas. Es ideal para quienes priorizan velocidad y simplicidad.

st, desarrollado por el proyecto *suckless*, adopta una filosofía radicalmente minimalista. Su configuración no se realiza mediante archivos externos, sino modificando directamente el código fuente escrito en C y recompilando el programa. Esto proporciona un control absoluto sobre el comportamiento del terminal, pero requiere conocimientos más profundos del sistema y del proceso de compilación.

En esta guía se utiliza Kitty como emulador de terminal principal debido a su equilibrio entre rendimiento, flexibilidad y facilidad de personalización. No obstante, el usuario puede seleccionar cualquiera de las opciones disponibles según sus preferencias y necesidades. La shell configurada en todos los casos continúa siendo Zsh, por lo que cambiar de Kitty a Alacritty o st no altera el funcionamiento de los comandos, alias o configuraciones definidas en archivos como `.zshrc`; únicamente modifica la aplicación que actúa como interfaz visual para interactuar con dicha shell.

Para cambiar el emulador de terminal predeterminado, es posible utilizar el atajo `Super + Alt + T`, que permite seleccionar entre las terminales instaladas en el sistema. A partir de este punto, la presente configuración se enfoca exclusivamente en Kitty y en la personalización detallada de sus archivos `kitty.conf` y `color.ini`, con el fin de obtener un entorno visual optimizado para productividad, desarrollo de software y administración avanzada del sistema. Por defecto se utiliza alacritty como terminal, para cambiarlo

Entonces hacemos

```
> super + alt + t
```

y seleccionar Kitty

A continuación para seleccionar y cambiar toda la parte de la terminal, en cuanto a Kitty

```
> cd ~/.config/kitty
> rm -rf *
```

Esto borra absolutamente todas las configuraciones anteriores de la Kitty, en mi caso las borro porque uso otra configuración.

```
> nvim kitty.conf

<> [Como no esta el archivo porque borramos todo, esto lo crea]
```

Y en el mismo archivo pegamos esto.

Importante

Se tiene que copiar antes, y en el modo insert (presionando `esc + i` dentro ya de nvim), que aparece abajo, tienes que presionar `p` para automáticamente que se pegue todo.

```
`enable_audio_bell no`

`include color.ini`

`font_family HackNerdFont`
`font_size 13`

`disable_ligatures never`

`url_color #61afef`

`url_style curly`

`map ctrl+left neighboring_window left`
`map ctrl+right neighboring_window right`
`map ctrl+up neighboring_window up`
`map ctrl+down neighboring_window down`

`map f1 copy_to_buffer a`
`map f2 paste_from_buffer a`
`map f3 copy_to_buffer b`
`map f4 paste_from_buffer b`

`cursor_shape beam`
`cursor_beam_thickness 1.8`
```

```
`mouse_hide_wait 3.0`  
`detect_urls yes`  
  
`repaint_delay 10`  
`input_delay 3`  
`sync_to_monitor yes`  
  
`map ctrl+shift+z toggle_layout stack`  
`tab_bar_style powerline`  
  
`inactive_tab_background #e06c75`  
`active_tab_background #98c379`  
`inactive_tab_foreground #000000`  
`tab_bar_margin_color black`  
  
`map ctrl+shift+enter new_window_with_cwd`  
`map ctrl+shift+t new_tab_with_cwd`  
  
`background_opacity 0.95`  
  
`shell zsh`
```

En el mismo modo, escribir simplemente en el teclado `:wq` para automáticamente guardar y salir

Ahora creamos el archivo `color.ini` con el mismo procedimiento

```
> nvim color.ini  
  
<> [Como no esta el archivo porque borramos todo, esto lo crea]
```

Y pegamos esto

```
cursor_shape          Underline  
cursor_underline_thickness 1  
window_padding_width  20  
  
# Special  
foreground #a9b1d6  
background #1a1b26  
  
# Black  
color0 #414868  
color8 #414868
```

```
# Red
color1 #f7768e
color9 #f7768e

# Green
color2 #73daca
color10 #73daca

# Yellow
color3 #e0af68
color11 #e0af68

# Blue
color4 #7aa2f7
color12 #7aa2f7

# Magenta
color5 #bb9af7
color13 #bb9af7

# Cyan
color6 #7dcfff
color14 #7dcfff

# White
color7 #c0caf5
color15 #c0caf5

# Cursor
cursor #c0caf5
cursor_text_color #1a1b26

# Selection highlight
selection_foreground #7aa2f7
selection_background #28344a
```

Para cambiar la configuración de la letra y que no se vea amontonado los iconos, instalamos jetbrains

```
> sudo pacman -S ttf-jetbrains-mono-nerd
> cd ~/.config/kitty
> nvim kitty.conf
```

En la línea 5, cambiar la letra de `font_family` a `JetBrainsMono Nerd Font`.
y el `font_size` a `12` o `13` dependiendo la configuración que se desea

Para habilitar el copy y paste entre la maquina y el OS principal

```
> cd /.config/bspwm
> nvim bspwmrc
<> [agregar en una linea]
> vmware-user-suid-wrapper &
```

Hay que borrar `fzf` para un buscador con `ctrl + t`:

```
> sudo pacman -R fzf
> git clone --depth 1 https://github.com/junegunn/fzf.git ~/.fzf
> ~/.fzf/install
>
<> [tanto como usuario no privilegiado como con root , (sudo su y repites)]
```

Configuraciones de iconos de la polybar

Todas las configuraciones de colores, tamaños e incluso de cada icono esta en la carpeta correspondiente del tema que estas ocupando:

```
> cd ~/.config/bspwm/rices/emilia
```

en la configuración de `modules.ini` y `config.ini` correspondientes, se encuentra absolutamente todas las configuraciones del entorno de cada uno de los temas. En el cual podras configurar a tu gusto el tamaño de cada uno de los módulos

Con nvim, busca `font-0`

Pantalla de bloqueo i3LockFancy

```
> git clone https://github.com/meskarune/i3lock-fancy.git
> cd i3lock-fancy
> sudo make install
```

Para configurar un atajo para i3lockfancy es necesario ver la ruta absoluta de donde lo descargaste -----> `which i3lock-fancy` copias la ruta, después vamos a `~/.config/bspwm/config` después `nvim sxhkdrc` y crear un modulo i3lock-fancy, agregando el atajo y la ruta absoluta, por ejemplo:

```
#LockScreen
ctrl + alt + x
/usr/bin/i3lock-fancy
```

`super + esc` para guardar cambios

Atajos para sudo

en el `.zshrc` con `cd .zshrc`

```
# Widget para alternar sudo
toggle_sudo() {
if [[ $BUFFER == sudo\ * ]]; then
BUFFER=${BUFFER#sudo }
else
BUFFER="sudo $BUFFER"
fi
CURSOR=${#BUFFER}
}

zle -N toggle_sudo

# Detectar doble ESC
esc_sequence() {
read -t 0.2 -k 1 key
if [[ $key == '\e' ]]; then
zle toggle_sudo
fi
}

zle -N esc_sequence
bindkey '\e' esc_sequence
```

Instalación de fzf (buscador)

Primero se tiene que eliminar el fzf, ya que el fzf que contiene es un poco antiguo a su vez la interfaz es mala, con el comando:

```
sudo pacman -R fzf
```

Eliminamos por completo el fzf, puede verificarse si se ejecuta directamente `fzf` y aparece `command not found`

```
git clone --depth 1 https://github.com/junegunn/fzf.git ~/.fzf
~/.fzf/install

echo 'export PATH="$HOME/.fzf/bin:$PATH"' >> ~/.zshrc
echo '[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh' >> ~/.zshrc
source ~/.zshrc
```

y en root hacer lo mismo

```
git clone --depth 1 https://github.com/junegunn/fzf.git ~/.fzf
~/.fzf/install

echo 'export PATH="$HOME/.fzf/bin:$PATH"' >> ~/.zshrc
echo '[ -f ~/.fzf.zsh ] && source ~/.fzf.zsh' >> ~/.zshrc
source ~/.zshrc
```

las primeras líneas de comando se pueden encontrar en la instalación del repositorio oficial de fzf

<https://github.com/junegunn/fzf>

Mientras que las ultimas 3 son para que el fzf se exporte y este habilitado no solo en bash, sino que también en zsh que es la Shell que estamos utilizando

Keybindings

```
ctrl + r
ctrl + t
```

Configuración de root inválida

```
> cp -r /home/nitblan/.config /root/
> cp -r /home/nitblan/.local /root/
> cp /home/nitblan/.bashrc /root/
> cp /home/nitblan/.zshrc /root/ 2>/dev/null
> chown -R root:root /root/.config /root/.local
> chsh -s /bin/zsh
```

Verificar con `echo $SHELL` y el root debería tener la misma configuración que el entorno original del usuario

Uso de Pacman, AUR (Arch user repositories) Helpers (Opcional)

En Arch Linux, el sistema de gestión de paquetes es uno de los componentes más importantes, ya que permite instalar, actualizar, buscar y eliminar software de manera eficiente y segura. El gestor de paquetes oficial es Pacman, una herramienta desarrollada específicamente para administrar tanto los paquetes binarios como las dependencias del sistema. Por ejemplo, al ejecutar el comando `sudo pacman -Syu`, Pacman sincroniza la base de datos de paquetes (`-Sy`), actualiza todo el sistema a las versiones más recientes (`-u`) y garantiza que las dependencias se mantengan consistentes. Su principal ventaja es que es el método oficial y más confiable para mantener el sistema, ya que obtiene software directamente de los repositorios mantenidos por la comunidad y los desarrolladores de Arch Linux, priorizando la estabilidad, la seguridad y la compatibilidad.

Además de Pacman, existen herramientas más avanzadas como Yay y Paru, conocidas como *AUR helpers*. Estas utilidades no reemplazan a Pacman, sino que lo extienden al permitir acceder también al Arch User Repository (AUR), un enorme repositorio mantenido por la comunidad que contiene miles de paquetes no incluidos en los repositorios oficiales. La gran ventaja de Yay es su facilidad de uso y su sintaxis casi idéntica a la de Pacman, lo que permite instalar paquetes oficiales y del AUR con comandos como `yay -S nombre_paquete`. Por su parte, Paru ofrece una interfaz más moderna, escrita en el lenguaje Rust Foundation Rust, con una salida más limpia, mejor manejo de dependencias y mayor control sobre la compilación de paquetes.

En términos de diferencias, Pacman es el gestor base y obligatorio en cualquier instalación de Arch Linux, enfocado exclusivamente en los repositorios oficiales y en la máxima confiabilidad. Yay destaca por su popularidad, simplicidad y rapidez de adopción, siendo ideal para usuarios que desean una experiencia práctica y familiar. Paru, en cambio, está orientado a quienes buscan una herramienta más sofisticada y con mejor presentación de información, ofreciendo una experiencia más detallada y profesional. En conjunto, estas herramientas convierten a Arch Linux en una de las distribuciones más flexibles y poderosas del ecosistema GNU/Linux, al permitir que el usuario tenga control total sobre el software instalado y sobre la forma en que administra su sistema.

Instalación de Paru

```
sudo pacman -S --needed base-devel git
```

Y dentro del directorio personal de trabajo en `/home/`

```
cd
git clone http://aur.archlinux.org/paru.git
cd paru
```

```
rustup default stable  
makepkg -si
```

Instalación de yay

```
cd  
git clone http://aur.archlinux.org/yay.git  
cd paru  
makepkg -si
```

Kitty Terminal - Atajos y Gestión de Ventanas

Crear nuevas terminales

Nueva terminal en el directorio actual

```
Ctrl + Shift + Enter
```

Crea una nueva ventana interna utilizando el mismo directorio de trabajo actual (`cwd`).

Configuración:

```
map ctrl + shift + enter new_window_with_cwd
```

Nueva pestaña

```
Ctrl + Shift + T
```

Abre una nueva pestaña utilizando el mismo directorio actual.

Configuración:

```
map ctrl + shift + t new_tab_with_cwd
```

Gestión de Layouts

Modo Stack (Pantalla Completa Alternable)

```
Ctrl + Shift + Z
```

Alterna entre el layout actual y el modo Stack.

Configuración:

```
map ctrl + shift + z toggle_layout stack
```

¿Qué hace?

Cuando existen varias terminales abiertas:

Modo normal:

```
+-----+  
| Terminal|  
+-----+  
| Terminal|  
+-----+
```

Modo Stack:

```
+-----+  
|       |  
|  Terminal 1  |  
|       |  
+-----+
```

Las demás terminales continúan ejecutándose en segundo plano y pueden recuperarse posteriormente.

Navegación entre ventanas

Ir a la ventana izquierda

```
Ctrl + ←
```

Configuración:

```
map ctrl + left neighboring_window left
```

Ir a la ventana derecha

Ctrl + →

Configuración:

```
map ctrl + right neighboring_window right
```

Ir a la ventana superior

Ctrl + ↑

Configuración:

```
map ctrl+up neighboring_window up
```

Ir a la ventana inferior

Ctrl + ↓

Configuración:

```
map ctrl+down neighboring_window down
```

Portapapeles Múltiples

Kitty permite mantener varios buffers de copiado independientes.

Buffer A

Copiar:

F1

Pegar:

F2

Configuración:

```
map f1 copy_to_buffer a
map f2 paste_from_buffer a
```

Buffer B

Copiar:

F3

Pegar:

F4

Configuración:

```
map f3 copy_to_buffer b
map f4 paste_from_buffer b
```

Caso de Uso para Pentesting

Ejemplo de flujo de trabajo:

Ventana 1:

```
nmap
```

Ventana 2:

```
ffuf
```

Ventana 3:

```
evil-winrm
```


Ventana 4:

```
nvim notas.md o un script.py
```

Al presionar:

```
Ctrl + Shift + Z
```

Todas las terminales siguen funcionando, pero sólo se visualiza una a la vez en pantalla completa, sin perder la anterior, al volver a presionar, se recupera.

Comando para conocer la versión de Kitty

```
kitty --version
```

Ubicación de Configuración

Archivo principal:

```
~/.config/kitty/kitty.conf
```

Archivo de colores:

```
~/.config/kitty/color.ini
```

Conceptos Importantes

Shell

Programa que interpreta comandos.

Ejemplos:

- bash
- zsh
- fish

Ver shell actual:

```
echo $SHELL
```